

Napredni modeli i baze podataka

Predavanja
Studen 2025.

3. Objektno-relacijske baze podataka

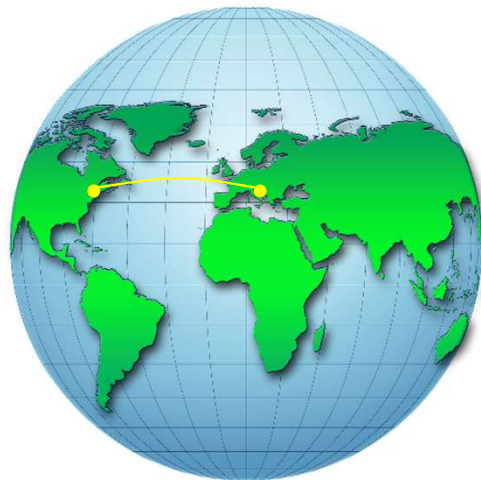


Pregled

- Zašto objektno-relacijski model podataka
- Objektno-relacijske mogućnosti prema SQL standardu
- Objektno-relacijska proširenja u PostgreSQL SUBP-u

Motivacijski primjer 1

Predstavljanje geografskih podataka u relacijskoj bazi podataka



- Za određivanje položaja točke na Zemlji koriste se geografske koordinate:
 - geografska širina (latitude)
 - geografska dužina (longitude)
- Udaljenost d u km između dvije geografske koordinate $(lat1, long1)$ i $(lat2, long2)$, uzevši u obzir zakrivljenost Zemlje, može se izračunati pomoću npr. Haversinove formule:

$$a = \sin^2\left(\frac{lat2-lat1}{2}\right) + \cos(lat1) * \cos(lat2) * \sin^2\left(\frac{long2-long1}{2}\right)$$

$$d = R * 2 * \operatorname{atan2}(\sqrt{a}, \sqrt{1-a}) \quad R = 6371 \text{ prosječna vrijednost radijusa Zemlje}$$

[Pogledaj tutorial sa zadacima za vježbu.](#)

Motivacijski primjer 1

Predstavljanje geografskih podataka u relacijskoj bazi podataka

Želim od SUBP

- ugrađen tip podatka (npr. point) za prezentaciju točke na Zemlji pomoću geografskih koordinata
- ugrađenu funkciju
 - za određivanje udaljenosti između dvije točke na Zemlji
 - koja će pomoću dvije zadane točke znati odrediti liniju koja ih spaja
 - koja će pomoću n zadanih točaka znati odrediti najkraći put koji spaja te točke redom kojim su navedene
 - koja će pomoću n zadanih točaka znati odrediti zatvoreni poligon čiji su vrhovi zadane točke
 - koja će znati izračunati površinu gornjeg poligona
 - ...
- da sve brzo radi 😊

Ništa od navedenog tradicionalni relacijski SUBP ne podržavaju

- točka je složeni tip podatka – sastoji se od 2 elementarna podatka
- linija kao skup točaka je također složeni tip podatka, jednako kao i poligon

Tradicionalni relacijski SUBP mora podržati nove složene tipove podataka i funkcije za rad s njima. Naravno, i nove vrste indeksiranja.

Motivacijski primjer 2

Predstavljanje i pretraživanje dokumenata u relacijskoj bazi podataka

Želim od SUBP

- ugrađen tip podatka pomoću kojeg će se
 - dokument predstaviti listom leksema te za svaki leksem pamti pozicije na kojima se pojavljuje u tekstu *
 - uvjet pretrage predstaviti listom leksema povezanih logičkim operatorom & ili || **
 - dokument predstaviti listom q-grama
- ugrađenu funkciju koja će
 - tekst na nekom svjetskom jeziku pretvoriti u ugrađeni tip podatka *
 - uvjet pretrage na nekom svjetskom jeziku pretvoriti u ugrađeni tip podatka **
 - odrediti sličnost između uvjeta i dokumenta temeljem morfologije, sintakse i semantike jezika
 - rangirati dokumente prema sličnosti između uvjeta i dokumenta
 - odrediti sličnost između uvjeta i dokumenta temeljem broja podudarnih q-grama
 - ...
- da sve brzo radi 😊

Motivacijski primjer 2

Predstavljanje i pretraživanje dokumenata u relacijskoj bazi podataka

Ništa od navedenog tradicionalni relacijski SUBP ne podržavaju

- lista leksema je složeni podatkovni tip: lista znakovnih nizova, a uz svaki znakovni niz dodatno je povezana lista pozicija na kojima se niz u tekstu nalazi
- lista q-grama također je složeni podatkovni tip
- tradicionalni načini indeksiranja (B-stablo) neće biti prikladni

Tradicionalni relacijski SUBP mora podržati:

- **nove složene tipove podataka**
- **funkcije za rad s njima**
- **nove načine indeksiranja složenih tipova podataka**
- ...

Objektno-relacijski sustav za upravljanje bazama podataka (ORDBMS)

- pokušaj spajanja najboljeg iz relacijskog i objektno-orijentiranog pristupa
- primjenjuje **objektno-relacijski model podataka** temeljen na relacijskom modelu podataka
 - očuvane karakteristike relacijskog modela
 - zadržana kompatibilnost s relacijskim upitnim jezikom
 - relacijski model proširen objektnim konceptima koji npr. omogućuju rukovanje novim tipovima podataka (npr. dozvoljene su složene vrijednosti u atributima)
 - uvećane mogućnosti modeliranja podataka čime je proširen opseg primjene
- SQL standardom predviđeni objektno-relacijski koncepti nisu potpuno implementirani niti u jednom SUBP-u
- U manjoj ili većoj mjeri objektno-relacijski koncepti ugrađeni su u sve poznatije RSUBP
- Primjeri na slajdovima - PostgreSQL

SQL standard: objektno-relacijska proširenja

- SQL:1999 uključuje većinu objektno-relacijskih koncepata
 - konstruktori složenih tipova, identitet objekta, učajurivanje, nasljeđivanje
- podjela tipova podataka:
 - **unaprijed definirani tipovi (*predefined types*)**
 - atomarni tip: integer, float, character, boolean, datetime ...
 - **izgrađeni tipovi (*constructed types*)**
 - izgrađeni atomarni tipovi (*constructed atomic types*)
 - referenca (*reference*)
 - izgrađeni kompozitni tipovi (*constructed composite types*)
 - **kolekcije (*collection*): polje (*array*), multiset**
 - *row*
 - **korisnički definirani tipovi (*user-defined types* - UDT)**
 - *distinct type*
 - **strukturirani tip (*structured type*)**

Primjer: Informacijski sustav studentske službe

- osoba (nije u 1NF)

osoba	sifOsoba	ime	prezime	adresa		slika	zivotopis
				ulica	mjesto		
					postBr nazMjesto		
	11001	Hrvoje	Novak	Ilica 25	10000 Zagreb		Rođen je
	78936	Ana	Kolar	Marmontova 18	21000 Split		

strukturirani
tip

strukturirani
tip

- predmet (nije u 1NF)

predmet	sifPred	nazPred	izvodjaci	polaznici
	1	Napredne baze podataka	{123, 111, 345, 24}	{13503, 14111, 14678, ...}

kolekcija

kolekcija

SQL standard: Kolekcija

Kolekcija vrijednosti homogenog podatkovnog tipa koji ne mora biti atomarni:

- polje (ARRAY)
 - uređena kolekcija; svaki element ima indeks i redoslijed elemenata je važan
- multiskup (MULTISET)
 - neuređena kolekcija; redoslijed elemenata nije važan

PostgreSQL: ARRAY (1)

```
CREATE TABLE tramLinija
(sifLinija SERIAL PRIMARY KEY,
 kratLinija CHAR(3) NOT NULL UNIQUE,
 nazivLinija VARCHAR(120) NOT NULL UNIQUE,
 linijaPostaje INTEGER[]); --ili
-- linijaPostaje INTEGER ARRAY[];
```

```
CREATE TABLE tramPostaja
(sifPostaja SERIAL PRIMARY KEY,
 nazivPostaja VARCHAR(120) NOT NULL
 UNIQUE);
```

```
INSERT INTO tramPostaja (nazivPostaja)
VALUES ('Zapadni kolodvor');
INSERT INTO tramPostaja (nazivPostaja)
VALUES ('Talovčeva');
```

...

```
...
linijaPostaje INTEGER[]
REFERENCES tramPostaja(sifPostaja)
...
```

- strani ključ (još) nije moguće definirati nad elementima polja,



PostgreSQL: ARRAY (2)

- upisivanje n-torke s atributom tipa ARRAY:

```
INSERT INTO tramLinija (kratLinija, nazivLinija, linijaPostaje)
VALUES ('1', 'Zapadni kolodvor - Borongaj',
        '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16}')
```

ili :

```
INSERT INTO tramLinija (kratLinija, nazivLinija, linijaPostaje)
VALUES ('2', 'Črnomerec - Savišće',
        ARRAY[17, 18, 19, 20, 3, 2, 21, 22, NULL, NULL, NULL])
```

ili :

```
INSERT INTO tramLinija (kratLinija, nazivLinija, linijaPostaje)
VALUES ('3', 'Ljubljana - Savišće',
        '{}') /* ili ARRAY[]::integer[] */
```

```
SELECT * FROM tramLinija
```

TRAMVAJSKE LINIJE TRAM ROUTES

- 1 ZAPADNI KOLODVOR - BORONGAJ
- 2 ČRNUMEREC - SAVIŠĆE
- 3 LJUBLJANICA - SAVIŠĆE
- 4 SAVSKI MOST - DUBEC

Redoslijed postaja
prati indeks
elementa polja.
Indeks ide od 1.
ako nije drugačije
zadano.

sifLinija integer	kratLinija character(3)	nazivLinija character varying(120)	linijaPostaje integer[]
1	1	Zapadni kolodvor – Borongaj	{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16}
2	2	Črnomerec - Savišće	{17, 18, 19, 20, 3, 2, 21, 22, NULL, NULL, NULL}
3	3	Ljubljana – Savišće	{}

PostgreSQL: ARRAY (3)

- upisivanje n-torke s atributom tipa ARRAY - elementi polja dobiveni SELECT naredbom:

```
INSERT INTO tramLinija (kratLinija, nazivLinija, linijaPostaje)
VALUES ('4', 'Savski most - Dubec',
        ARRAY (SELECT sifPostaja FROM tramPostaja
                  WHERE sifPostaja BETWEEN 56 AND 70)
        )
```

- pristupanje elementima polja

korištenjem
indeksa elementa

```
SELECT nazivLinija,
       linijaPostaje[1] AS prva,
       linijaPostaje[2] AS druga,
       linijaPostaje[array_length (linijaPostaje, 1)] AS zadnja
FROM tramLinija
WHERE linijaPostaje[2] = 2 OR linijaPostaje[1:2] = ARRAY [17,18]
```

array_length (polje,dim) –
broj elemenata u polju
(za zadanu dimenziju dim)

pristupanje elementima
čiji su indeksi unutar
navedenih granica

nazivLinija character varying(120)	prva integer	druga integer	zadnja integer
Zapadni kolodvor – Borongaj	1	2	16
Črnomerec - Savišće	17	18	

PostgreSQL: ARRAY (4)

Operator	Opis	Primjer
=, <>	usporedba	ARRAY[1,2,3] = ARRAY[1,2,3] → true ARRAY[1,2,3,4] = ARRAY[1,2,4,3] → false
@>	sadrži	ARRAY[3,1,4,2] @> ARRAY[2,1] → true ARRAY[3,1,4,2] @> ARRAY[4,4] → true
<@	sadržan je u	ARRAY[3,2] <@ ARRAY[4,7,1,2,3] → true
&&	imaju li polja zajedničke elemente	ARRAY[4,3] && ARRAY[2,1,4] → true ARRAY[4,3] && ARRAY[2,1] → false
	konkatenacija	ARRAY[1,2] 3 → {1,2,3} ARRAY[1,2] ARRAY[3,4] → {1,2,3,4}

Funkcija	Opis	Primjer
array_length (<i>polje</i> , <i>dim</i>)	broj elemenata u polju (za zadanu dimenziju <i>dim</i>)	array_length(ARRAY[1,2,3], 1) → 3
array_lower (<i>polje</i> , <i>dim</i>) array_upper (<i>polje</i> , <i>dim</i>)	donja/gornja granica (indeks) za zadanu dimenziju polja	array_lower(ARRAY[0,1,8], 1) → 1 array_upper(ARRAY[0,1,8], 1) → 3 array_lower('[-5:-2]={0, 1,2,3}'::integer[], 1) → -5 array_upper('[-5:-2]={0, 1,2,3}'::integer[], 1) → -2
array_append (<i>polje</i> , <i>el</i>) array_prepend (<i>el</i> , <i>polje</i>)	dodavanje elementa na kraj/početak polja	array_append(ARRAY[1,2], 3) → {1,2,3} array_prepend(3, ARRAY[1,2]) → {3,1,2}
array_remove (<i>polje</i> , <i>el</i>)	brisanje elemenata vrijednosti <i>el</i> iz jednodimenzionalnog polja	array_remove(ARRAY[3,1,3,2], 3) → {1,2}

PostgreSQL: motivacijski primjer - odgnježđivanje

Za tramvajsku liniju s kraticom „1” potrebno je ispisati redni broj i naziv postaje od polazne do završne. Poredati ih prema redoslijedu postaja silazno.

tramLinija	sifLinija integer	kratLinija character(3)	nazivLinija character varying(120)	linijaPostaje integer[]
	1	1	Zapadni kolodvor – Borongaj	{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16}
	2	2	Črnomerec - Savišće	{17, 18, 19, 20, 3 , 2, 21, 22, NULL, NULL, NULL}
	3	3	Ljubljana – Savišće	{}

tramPostaja	sifPostaja integer	nazivPostaja varchar(120)
	1	Zapadni kolodvor
	2	Talovčeva
	3	Reljkovićeve
	...	...

- Tramvajske postaje na liniji su pohranjene u kolekciji ***linijaPostaje INTEGER ARRAY[]***.
- Naziv postaje treba dobiti spajanjem vrijednosti svakog elementa cjelobrojnog polja s relacijom ***tramPostaja***.
- Sortiranje treba obaviti prema indeksu elemenata polja.

Kako to izvesti u SQL-u?

PostgreSQL: motivacijski primjer -odgnježđivanje

Za tramvajsku liniju s kraticom „1” potrebno je ispisati redni broj i naziv postaje od polazne do završne. Poredati ih prema redoslijedu postaja silazno.

tramLinija

sifLinija integer	kratLinija character(3)	nazivLinija character varying(120)	linijaPostaje integer[]
----------------------	----------------------------	---------------------------------------	-----------------------------

tramPostaja

sifPostaja integer	nazivPostaja varchar(120)
-----------------------	------------------------------

U relacijskom modelu kolekcija **linijaPostaje** bila bi predstavljena relacijom **linijaPostaja**.

tramLinija

<u>sifLinija</u>	kratLinija	nazivLinija
------------------	------------	-------------

tramPostaja

<u>sifPostaja</u>	nazivPostaja
-------------------	--------------

linijaPostaja

<u>sifLinija</u>	<u>sifPostaja</u>	rbrPostaja
------------------	-------------------	------------

Rješenje bi bilo trivijalno:

```
SELECT rbrPostaja, nazivPostaja
FROM tramPostaja
NATURAL JOIN linijaPostaja
NATURAL JOIN tramLinija
WHERE kratLinija = '1'
ORDER BY rbrPostaja DESC
```

Ideja: kolekciju **tramLinija.linijaPostaje** transformirati u „privremenu” relaciju i koristiti u SQL upitima na uobičajeni način.

PostgreSQL: odgnježdivanje

Funkcija	Opis	Primjer
<code>unnest (polje) [WITH ORDINALITY] [AS] table_alias [(column_alias [, ...])]</code>	kreira jednu n-torku za svaki element polja Uz specificiran WITH ORDINALITY, generira se redni broj elementa polja počevši od 1.	<code>unnest(ARRAY[3,2])</code> → dvije n-torke: 3 2

tramLinija

sifLinija

kratLinija

nazivLinija

linijaPostaje
Integer[]

tramPostaja

sifPostaja

nazivPostaja

```
SELECT nazivLinija,  
       UNNEST(linijaPostaje) AS sifPostaja  
FROM tramLinija  
WHERE kratLinija = '1'
```

ili

```
SELECT nazivLinija,  
       linijaPostaja.sifPostaja  
FROM tramLinija,  
     UNNEST(linijaPostaje) AS linijaPostaja (sifPostaja)  
WHERE kratLinija = '1'
```

nazivLinija	sifPostaja
Zapadni kolodvor – Borongaj	1
Zapadni kolodvor – Borongaj	2
...	...
Zapadni kolodvor – Borongaj	16

PostgreSQL: odgnježdvanje

Za tramvajsku liniju s kraticom „1” potrebno je ispisati redni broj i naziv postaje od polazne do završne. Poredati ih prema redoslijedu postaja silazno.

tramLinija

<u>sifLinija</u>	kratLinija	nazivLinija	linijaPostaje Integer[]
------------------	------------	-------------	-----------------------------

tramPostaja

sifPostaja	nazivPostaja
------------	--------------

```
SELECT linijaPostaja.rbrPostaja,  
       tramPostaja.nazivPostaja  
FROM tramLinija,  
     UNNEST(tramLinija.linijaPostaje) WITH ORDINALITY  
     AS linijaPostaja (sifPostaja, rbrPostaja),  
     tramPostaja  
WHERE tramPostaja.sifPostaja = linijaPostaja.sifPostaja  
      AND kratLinija = '1'  
ORDER BY rbrPostaja DESC
```

rbrPostaja	nazivPostaja
16	Borongaj
15	Svetice
14	Harambašićeva
...	...
2	Talovčeva
1	Zapadni kolodvor

PostgreSQL: odgnježdjivanje

Za tramvajsku liniju s kraticom "1" ispisati redni broj postaje i njen naziv ako u nazivu sadrži riječ **nalik** riječi "kolodver". Nalikuje li riječ iz naziva riječi "kolodver" ispitajte pomoću implementacije Q-gram algoritma u PostgreSQL-u.

tramLinija

<u>sifLinija</u>	kratLinija	nazivLinija	linijaPostaje Integer[]
------------------	------------	-------------	-----------------------------

tramPostaja

<u>sifPostaja</u>	nazivPostaja
-------------------	--------------

1. Izdvojiti riječi u nazivu postaje

```
SELECT sifPostaja,  
       string_to_array(nazivPostaja, ' ') rijeci  
FROM tramPostaja
```

Izdvaža riječi odvojene razmakom.
Rezultat je array.

sifPostaja integer	rijeci text[]
1	{Zapadni, kolodvor}
...	...
4	{Trg, dr., Franje, Tuđmana}
...	...

2. Odgnijezditi array, sačuvati pripadnost riječi tram. postaji

```
SELECT sifPostaja,  
       UNNEST (string_to_array(nazivPostaja, ' ')) rijec  
FROM tramPostaja
```

sifPostaja integer	rijec text
1	Zapadni
1	kolodvor
...	...
4	Trg
...	...

3. Riječ nalikuje riječi "kolodver"

```
SELECT *  
FROM (SELECT sifPostaja,  
            UNNEST (string_to_array(nazivPostaja, ' ')) rijec  
      FROM tramPostaja) rijeciTP  
WHERE rijeciTP.rijec % 'kolodver'
```

sifPostaja integer	rijec text
1	kolodvor
...	...

PostgreSQL: odgnježđivanje

Za tramvajsku liniju s kraticom "1" ispisati redni broj i naziv postaje koja u nazivu sadrži riječ **nalik** riječi "kolodver". Nalikuje li riječ iz naziva riječi "kolodver" ispitajte pomoću implementacije Q-gram algoritma u PostgreSQL-u.

tramLinija

<u>sifLinija</u>	kratLinija	nazivLinija	linijaPostaje Integer[]
------------------	------------	-------------	-----------------------------

tramPostaja

<u>sifPostaja</u>	nazivPostaja
-------------------	--------------

```
SELECT linijaPostaja.rbrPostaja,  
       tramPostaja.nazivPostaja,  
       rijeciTP.rijec  
FROM   tramLinija,  
       UNNEST(tramLinija.linijaPostaje) WITH ORDINALITY  
         AS linijaPostaja (sifPostaja, rbrPostaja),  
       tramPostaja,  
       UNNEST (string_to_array(tramPostaja.nazivPostaja, ' '))  
         AS rijeciTP (rijec)  
WHERE  tramPostaja.sifPostaja = linijaPostaja.sifPostaja  
AND    kratLinija = '1'  
       AND rijeciTP.rijec % 'kolodver'  
ORDER BY rbrPostaja DESC
```

rbrPostaja	nazivPostaja	rijec
1	Zapadni kolodvor	kolodvor

PostgreSQL: motivacijski primjer - ugnježđivanje

Relaciju **tramPostaja** potrebno je proširiti atributom **tramLinije** u kojem će se u obliku kolekcije pohraniti nazivi svih linija koje kroz postaju prolaze. Atribut ažurirati u skladu s podacima u relaciji **tramLinija**.

tramPostaja

<u>sifPostaja</u>	<u>nazivPostaja</u>	<u>tramLinije</u> character varying(120) []
1	Zapadni kolodvor	{"Zapadni kolodvor - Borongaj"}
2	Talovčeva	{"Zapadni kolodvor - Borongaj", "Črnomerec - Savišće"}
...	...	

tramLinija

<u>sifLinija</u>	<u>kratLinija</u>	<u>nazivLinija</u>	<u>linijaPostaje</u> Integer []
------------------	-------------------	--------------------	-------------------------------------

```
ALTER TABLE tramPostaja ADD tramLinije VARCHAR(120) [ ];
```

```
UPDATE tramPostaja
SET tramLinije =
(SELECT nazivLinija
 FROM tramLinija
 WHERE tramLinija.linijaPostaje...tramPostaja.sifPostaja)
```

1. operator „sadrži”

2. Int pretvoriti u polje (da bi oba operanda bila polja)

3. Relacija koju treba pretvoriti u polje nizova znakova

1. Operator „sadrži” je @>

2. Pretvorba int u polje dobije se pomoću ARRAY [sifPostaja]

```
SELECT ARRAY[sifPostaja]
FROM tramPostaja
```

array

Integer []

{1}

{2}

...

PostgreSQL: ugnježdživanje

tramPostaja

sif
Postaja

naziv
Postaja

tramLinije
character varying(120) []

```
UPDATE tramPostaja
SET tramLinije =
(SELECT nazivLinija
 FROM tramLinija
 WHERE tramLinija.linijaPostaje ... tramPostaja.sifPostaja)
```

3. pretvoriti u polje znakovnih nizova

tramLinija

<u>sif</u> Linija	krat Linija	naziv Linija	linijaPostaje Integer[]
----------------------	----------------	-----------------	-----------------------------

3. Polje možemo dobiti sa ARRAY

```
SELECT ARRAY[nazivLinija]
FROM tramLinija
WHERE linijaPostaje @> ARRAY[3]
```

```
UPDATE tramPostaja
SET tramLinije =
(SELECT ARRAY[nazivLinija]
 FROM tramLinija
 WHERE linijaPostaje @>
ARRAY[tramPostaja.sifPostaja])
```

korelirani podupit

ERROR: more than one row returned by a subquery used as an expression

array bpchar []

{"Zapadni kolodvor - Borongaj"}

{"Črnomerec - Savišće"}



pretvoriti u polje
znakovnih nizova

array

character varying(120) []

{"Zapadni kolodvor - Borongaj",
"Črnomerec - Savišće"}

Podupit mora vratiti jednu n-torku, a ne 2 ili više!

Trebam „nešto”, npr. funkciju, koja će više zasebnih elemenata polja transformirati u kolekciju.

PostgreSQL: ugnježdivanje

Funkcija	Opis
<code>array_agg (izraz)</code>	kreira polje čiji su elementi n-torke – predane argumentom, uključujući NULL vrijednost (vraća NULL vrijednost, ako nije dohvaćena niti jedna n_torka)

```
UPDATE tramPostaja
SET tramLinije =
(SELECT array_agg (nazivLinija)
FROM tramLinija
WHERE tramLinija.linijaPostaje @> ARRAY[sifPostaja])
```

```
SELECT *
FROM tramPostaja;
```

sif Postaja	nazivPostaja	linije character varying(120) []
1	Zapadni kolodvor	{"Zapadni kolodvor - Borongaj"}
2	Talovčeva	{"Zapadni kolodvor - Borongaj", "Črnomerec - Savišće"}

Rezultat *izraza* (argumenta funkcije `array_agg`) moraju biti n-torke s jednim atributom!
Što ako želim pored naziva linije u novi atribut spremiti i njenu kraticu?

```
SELECT ARRAY_agg[kratLinija, nazivLinija]
FROM tramLinija
WHERE linijaPostaje @> ARRAY[3]
```

ERROR: syntax error at or near "," LINE 1: SELECT ARRAY_agg[kratLinija, nazivLinija]

Je li moguće da PostgreSQL ima toliko skromnu podršku za ugnježdivanje?
Nije. Elementi kolekcije mogu biti složenog tipa.

SQL standard: Strukturirani tipovi

- imenovani, korisnički-definiran podatkovni tip
- proizvoljno složene strukture
- atribut može biti strukturiranog tipa
 - navodi se ime i podatkovni tip
- primjer: definicija tipa *mjestoT*

```
CREATE TYPE mjestoT AS (  
    postBr      INTEGER,  
    nazMjesto   VARCHAR(40))  
;
```

- strukturirani tipovi se mogu koristiti kao:
 - tip atributa drugog strukturiranog tipa
 - tip parametra funkcija, metoda i procedura
 - tip SQL varijable
 - tip atributa i tip n-torke relacije
- **tipizirana tablica** (*typed table*) – shema tablice definirana pomoću strukturiranog tipa
 - atributi tipa postaju stupci tablice

```
CREATE TABLE mjesto OF mjestoT  
(PRIMARY KEY (postBr));
```

PostgreSQL: strukturirani tip

- za atribut strukturiranog tipa se navodi naziv i podatkovni tip; nije moguće definiranje integritetskih ograničenja (npr. NOT NULL, CHECK, DEFAULT)
- atributi strukturiranog tipa mogu biti bilo kojeg tipa (uključujući druge složene tipove i ARRAY)
- kreiranje tipa:
 - naredbom CREATE TYPE

```
CREATE TYPE typeName AS ( atribName atribType [, ... ] )
```

npr:

```
CREATE TYPE drzavaT AS (oznDrzava CHAR(2)  
                        , nazDrzava VARCHAR(20)) ;
```

- automatski prilikom kreiranja tablice

```
CREATE TABLE drzava (oznDrzava CHAR(2)  
                     , nazDrzava VARCHAR(20)) ;
```

→ kreiran je
i tip *drzava*

PostgreSQL: strukturirani tip

- kreirani tip moguće je koristiti prilikom kreiranja:

- drugog tipa (atribut može biti složenog tipa)

```
CREATE TYPE drzavaT AS  
(oznDrzava CHAR(2),  
nazDrzava  
VARCHAR(20));
```

```
CREATE TYPE mjestoT AS  
(postBroj INTEGER,  
nazMjesto VARCHAR(20),  
drzava drzavaT);
```

- relacije (kao tip atributa relacije)

```
CREATE TABLE poduzece (naziv CHAR(20),  
sjediste mjestoT);
```

- Unos n-torke:

```
INSERT INTO poduzece  
VALUES ('INA',  
( 10000, 'Zagreb',  
( 'HR', 'Hrvatska' )  
)  
);
```

- pristup atributima složenog tipa - *dot* notacija

```
SELECT sjediste,  
(sjediste).postBroj,  
(sjediste).drzava,  
(sjediste).drzava.nazDrzava  
FROM poduzece;
```

sjediste mjestoT	postBroj integer	drzava drzavaT	nazDrzava varchar(20)
(10000,Zagreb,"(HR,Hrvatska)")	10000	(HR,Hrvatska)	Hrvatska

PostgreSQL: primjer – ugnježđivanje (složeni tip)

Relaciju **tramPostaja** potrebno je proširiti atributom **tramLinije** u kojem će se u obliku kolekcije pohraniti šifre, kratice i nazivi svih linija koje kroz postaju prolaze. Atribut ažurirati u skladu s podacima u relaciji **tramLinija**.

tramPostaja

<u>sifPostaja</u>	nazivPostaja	tramLinije tramLinijeT []
1	Zapadni kolodvor	{{1, "1", "Zapadni kolodvor - Borongaj"}}
2	Talovčeva	{{1, "1", "Zapadni kolodvor - Borongaj"}, (2, "2", "Črnomerec - Savišće")}
...	...	...

tramLinija

<u>sifLinija</u>	kratLinija	nazivLinija	linijaPostaje Integer[]
------------------	------------	-------------	-----------------------------

1. Kreirati novi složeni tip tramLinijaT:

```
CREATE TYPE tramLinijaT AS  
(sifLinija INTEGER,  
 kratLinija CHAR(3),  
 nazivLinija VARCHAR(120));
```

2. Dodati atribut tipa ARRAY s elementima složenog tipa tramLinijaT:

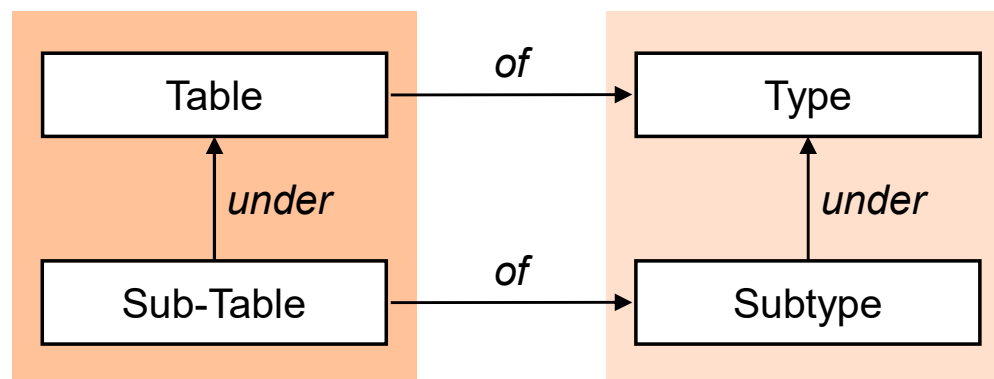
```
ALTER TABLE tramPostaja  
ADD tramLinije tramLinijaT[];
```

3. Ažurirati tramPostaja.tramLinije:

```
UPDATE tramPostaja  
SET tramLinije =  
(SELECT array_agg((sifLinija, kratLinija, nazivLinija)::tramLinijaT)  
FROM tramLinija  
WHERE tramLinija.linijaPostaje @> ARRAY[tramPostaja.sifPostaja])
```

SQL standard: Nasljeđivanje

- nasljeđivanje tipova
 - moguće je samo za korisnički definirane strukturirane tipove
 - hijerarhija tipova
- nasljeđivanje tablica
 - moguće je samo za tipizirane tablice
 - hijerarhija tablica



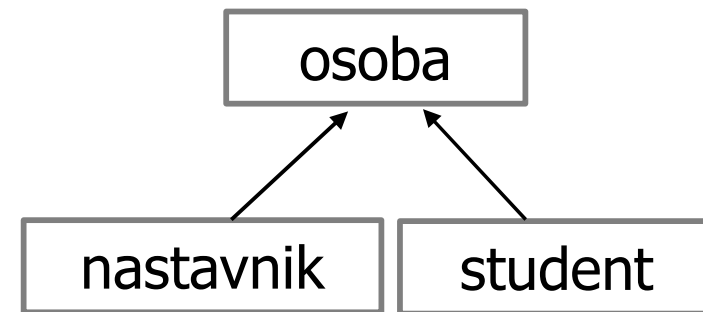
PostgreSQL: Nasljeđivanje (1)

- omogućeno je samo nasljeđivanje (temeljnih) tablica (ne i složenih tipova)
 - nasljeđuju se:
 - osnovne definicije atributa (naziv, tip, NULL ograničenje)
 - pretpostavljene (*default*) vrijednosti atributa
 - CHECK ograničenja (ne mogu biti nadjačana u tablicama djece)
 - metode tablica
 - ne nasljeđuju se:
 - indeksi
 - UNIQUE, PRIMARY KEY, FOREIGN KEY ograničenja
 - okidači

PostgreSQL: Nasljeđivanje

Primjer:

```
CREATE TABLE osoba(  
    sifra          INTEGER PRIMARY KEY,  
    ime            VARCHAR(25),  
    prezime        VARCHAR(25)) ;
```



```
CREATE TABLE nastavnik (  
    brTekRacun     CHAR(11),  
    zaposlenOd     DATE NOT NULL,  
    zaposlenDo     DATE  
) INHERITS (osoba);
```

```
CREATE TABLE student (  
    JMBAG          CHAR(10),  
    datumUpis      DATE  
) INHERITS (osoba);
```

student i nastavnik od osoba nasljeđuju attribute sifra, ime i prezime

```
INSERT INTO osoba      VALUES (1, 'Ana' , 'Ban');  
INSERT INTO student    VALUES (2, 'Mia' , 'Nel', '0036000001', '01.07.2013');  
INSERT INTO student    VALUES (3, 'Ivo' , 'Puž', '0036000002', '21.07.2014');  
INSERT INTO nastavnik  VALUES (4, 'Petar', 'Par', '23456187902', '01.05.2001', NULL);
```

```
SELECT * FROM osoba;
```

```
SELECT * FROM ONLY osoba;
```

```
SELECT * FROM nastavnik;
```

sifra	ime	prezime
1	Ana	Ban
2	Mia	Nel
3	Ivo	Puž
4	Petar	Par

sifra	ime	prezime
1	Ana	Ban

sifra	ime	prezime	brTekRacun	datumOd	datumDo
4	Petar	Par	23456187902	01.05.2001	

PostgreSQL: Nasljeđivanje

Problemi s primarnim ključem (UNIQUE ograničenjem, indeksima)

- podtablica ne nasljeđuje primarni ključ
- podtablica može sadržavati duplikat vrijednosti primarnog ključa iz nadtablice

```
CREATE TABLE osoba (  
    sifra            INTEGER PRIMARY KEY,  
    ime             VARCHAR(25),  
    prezime         VARCHAR(25)) ;
```

```
CREATE TABLE student (  
    JMBAG           CHAR(10),  
    datumUpis       DATE  
) INHERITS (osoba);
```

```
INSERT INTO osoba      VALUES (1, 'Ana'   , 'Ban');  
INSERT INTO osoba      VALUES (1, 'Tena'  , 'Pale'); → ERROR
```

```
INSERT INTO student    VALUES (1, 'Mia'   , 'Nel', '0036000001', '01.07.2013'); → OK  
INSERT INTO student    VALUES (1, 'Ivo'   , 'Puž', '0036000002', '12.07.2014'); → OK
```

```
SELECT * FROM osoba;
```

sifra	ime	prezime
1	Ana	Ban
1	Mia	Nel
1	Ivo	Puž

```
ALTER TABLE student ADD CONSTRAINT studentPk PRIMARY KEY (sifra);
```

Je li problem riješen?

Vidi u tutorialu (zadacima za vježbu) kako se problem može riješiti.

PostgreSQL: Nasljeđivanje

Problemi sa stranim ključem

1. strani ključevi se ne nasljeđuju

```
CREATE TABLE skola(  
    sifSkola    INTEGER PRIMARY KEY,  
    nazivSkola VARCHAR(250));
```

```
CREATE TABLE osoba(  
    sifra        INTEGER PRIMARY KEY,  
    ime          VARCHAR(25),  
    prezime      VARCHAR(25),  
    sifSkola     INTEGER REFERENCES  
                    skola(sifSkola));
```

```
CREATE TABLE student (  
    JMBAG        CHAR(10),  
    datumUpis    DATE  
) INHERITS (osoba);
```

```
INSERT INTO skola VALUES (1, 'Gimnazija');
```

```
INSERT INTO osoba    VALUES (1, 'Ana', 'Ban', 2);
```

→ **ERROR**

```
INSERT INTO student VALUES (2, 'Mia', 'Nel', 2, '0036000001', '01.07.2013'); → OK
```

Rješenje:

```
ALTER TABLE student ADD CONSTRAINT studentSkolaFk  
    FOREIGN KEY (sifSkola) REFERENCES skola(sifSkola);
```

Prethodno treba obrisati n-torku koja ne zadovoljava referencijski integritet.

PostgreSQL: Nasljeđivanje

Problemi sa stranim ključem - nastavak

2. nemogućnost stvaranja stranih ključeva koji se referenciraju na sve n-torke iz hijerarhije (iz nadtablice i svih njezinih podtablica)

- u obzir se uzimaju samo n-torke koja sudjeluje u definiciji stranog ključa kao referencirana – ne i druge tablice iz hijerarhije

```
CREATE TABLE osoba (  
    sifra            INTEGER PRIMARY KEY,  
    ime             VARCHAR(25),  
    prezime         VARCHAR(25));
```

```
CREATE TABLE student (  
    JMBAG           CHAR(10),  
    datumUpis       DATE  
) INHERITS (osoba);
```

```
CREATE TABLE ulazakUDvoranu (  
    sifra            INTEGER CONSTRAINT fkUUDOsoba REFERENCES osoba(sifra),  
    dvorana          VARCHAR(10),  
    trenutak         TIMESTAMP);
```

```
INSERT INTO osoba      VALUES (1, 'Ana' , 'Ban');  
INSERT INTO student    VALUES (2, 'Mia' , 'Nel', '0036000001', '01.07.2013');  
  
INSERT INTO ulazakUDvoranu VALUES (1, 'B5', '2016-10-22 10:50'); → OK  
INSERT INTO ulazakUDvoranu VALUES (2, 'B5', '2016-10-22 10:50'); → ERROR
```

ERROR: insert or update on table "ulazakudvoranu" violates foreign key constraint fkUUDOsoba"
DETAIL: Key (sifra)=(2) is not present in table "osoba".

Literatura

- J. A. Hoffer, R. Venkataraman, H.Topi: **Modern Database Management (11th Edition)**, Prentice Hall, 2013
 - Chapter 13: Overview: Object-Oriented Data Modeling
http://wps.prenhall.com/wps/media/objects/14735/15089538/M13_HOFF2253_11_SE_C13WEB.pdf
- Pearson Education: **Advanced Database Topics: Object-Relational Databases**
 - http://wps.pearsoned.co.uk/wps/media/objects/10977/11240737/Web%20chapters/Chapter%2014_WEB.pdf
- S.W. Dietrich, S.D. Urban: **An Advanced Course in Database Systems : Beyond Relational Databases**, Prentice Hall, 2005